

Warehouse Delivery Robot

Search Algorithms: BFS, UCS and A*

Intelligent Systems | Task 1

Member	Name	ID
Member 1	ريتاج اشرف عبدالحميد مغربي	2403247504
Member 2	كيرلس مجدي فخري	2403247848
Member 3	يوسف ايهاب محمد	2403244266

GitHub: <https://github.com/kirogames0/Intelligent-task.git>

1. Problem Formulation

A warehouse robot must travel from the start cell **S** to the goal cell **G** on a grid map. It moves one cell at a time (Up, Down, Left or Right) and cannot enter a wall. The task is to find a path from S to G using search algorithms, and to compare an uninformed method with an informed one. Three maps are used: MAP_A (a normal case), MAP_B (a trap: the shortest path is not the cheapest) and MAP_C (no solution).

Search problem definition

Component	Definition
State	The robot's position (row, column) on the grid.
Initial state	The position of S, found by scanning the grid.
Actions	Move Up, Down, Left or Right to a neighbouring cell that exists on the map and is not a wall.
Transition model	Moving into a neighbouring cell places the robot in that cell.
Step cost	Cost of entering the new cell: $\cdot = 1$, $\sim = 3$, G = 1. S is never entered, so it costs 0.
Goal test	The current position equals the position of G.
Path cost	The sum of the step costs along the path (S itself is not counted).

Map symbols

Symbol	Meaning	Cost of entering
S	Start cell	0 (not counted)
G	Goal cell	1
$\cdot$	Normal floor	1
$\sim$	Rough floor	3
#	Wall	cannot enter

Map representation

Each map is a list of strings. It is converted into a dictionary $\{(\text{row}, \text{col}): \text{symbol}\}$. A position that is off the map is simply not a key, so checking whether a neighbour exists is just `nb in cells`. Walls are left out of the cost table `COST`, so any symbol not in the table is impassable and no special wall code is needed.

2. Algorithm Descriptions

All algorithms are implemented from scratch. Only `heapq` (priority queue) and `collections.deque` (FIFO queue) are used as data structures; no path-finding library is used.

2.1 Breadth-First Search (BFS), uninformed

BFS keeps a FIFO queue and expands cells in order of the number of moves from S. A visited set prevents revisiting cells, and parent links are used to rebuild the path. BFS finds the path with the **fewest moves**, but it ignores terrain costs, so the path it returns is not always the cheapest. Its cost is computed afterwards by summing the entering cost of each cell on the path.

2.2 Uniform-Cost Search (UCS), uninformed

UCS keeps a priority queue ordered by the cost so far, $g(n)$. It always expands the cheapest known cell, so the first time G is removed from the queue, its path is the **cheapest** one. A dictionary `best_cost` stores the cheapest known cost to each cell. When a cheaper route to a cell is found, a new entry is pushed, and older entries with a higher cost are skipped when they are popped (stale-entry check).

2.3 A* Search, informed

A* works like UCS but orders the priority queue by $f(n) = g(n) + h(n)$, where $h(n)$ is a heuristic estimate of the remaining cost to G. The heuristic pulls the search toward the goal, so A* skips many cells that UCS has to explore, while still returning the cheapest path.

2.4 Heuristic: Manhattan distance

$h(n) = |\text{row}(n) - \text{row}(G)| + |\text{col}(n) - \text{col}(G)|$

- **Admissible:** every move into a cell costs at least 1, and the robot needs at least as many moves as the Manhattan distance (walls can only make the real route longer). So h never overestimates the true remaining cost, which guarantees that A* returns the cheapest path.
- **Consistent:** one move changes h by at most 1 and costs at least 1, so $h(n) \leq \text{cost}(n, n') + h(n')$. This is why A* can safely skip a cell once it has been expanded with its best cost.
- It is cheap to compute and suits a 4-direction grid.

2.5 Metrics recorded

For every run the program records: **path length** (number of moves), **path cost**, **nodes expanded** (cells removed from the frontier and processed), **maximum frontier size** (peak size of the queue, a measure of memory use), and **run time** in milliseconds measured with `time.perf_counter()`. If G cannot be reached, the program prints "No path found".

3. Results

Every algorithm was run on MAP_A, MAP_B and MAP_C. The table shows the results; run time is the time of a single run.

Map	Algorithm	Path length	Path cost	Nodes expanded	Max frontier	Time (ms)
MAP_A	BFS	11	15	28	5	0.0489
MAP_A	UCS	11	15	28	4	0.0585
MAP_A	A*	11	15	21	5	0.0471
MAP_B	BFS	8	22	25	4	0.0337
MAP_B	UCS	12	12	30	4	0.0503
MAP_B	A*	12	12	15	7	0.0315
MAP_C	BFS	-	-	13	3	0.0166
MAP_C	UCS	-	-	13	3	0.0220
MAP_C	A*	-	-	13	3	0.0237

MAP_C is marked "-" because no path exists. All three algorithms explore all 13 reachable cells before stopping. Single-run times are only a few hundredths of a millisecond and change slightly from run to run, so the node counts are the reliable measure of search effort.

Key observations

- **MAP_A:** all three find a path of cost 15 (11 moves). A* expands 21 nodes against 28 for BFS and UCS.
- **MAP_B (the trap):** BFS returns the 8-move path across the rough floor with cost **22**. UCS and A* return the 12-move path with cost **12**. A* expands 15 nodes, half of UCS's 30.
- **MAP_C:** no solution, so every algorithm correctly reports "No path found".

4. Path Screenshots

The output printed by the program for every algorithm on every map. In the drawn map, * marks the cells of the path found, **S** is the start, **G** is the goal, ~ is rough floor and # is a wall.

MAP A

All three algorithms reach G with cost 15; only the number of expanded nodes differs.

```
Algorithm : BFS
Path      : [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (3,
              7), (4, 7)]
Length    : 11 moves
Cost      : 15
Expanded  : 28
Max frontier: 5
Time      : 0.0489 ms
Map with path:
S**#....
***#.#.#.
..*****
.####.#*
...~..#G
```

```
Algorithm : UCS
Path      : [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (3,
              7), (4, 7)]
Length    : 11 moves
Cost      : 15
Expanded  : 28
Max frontier: 4
Time      : 0.0585 ms
Map with path:
S**#....
***#.#.#.
..*****
.####.#*
...~..#G
```

```
Algorithm : A*
Path      : [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (3,
              7), (4, 7)]
Length    : 11 moves
Cost      : 15
Expanded  : 21
Max frontier: 5
Time      : 0.0471 ms
Map with path:
S**#....
***#.#.#.
..*****
.####.#*
...~..#G
```

MAP B

BFS crosses the rough floor (8 moves, cost 22). UCS and A* take the longer normal-floor detour (12 moves, cost 12).

```
Algorithm : BFS
Path      : [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (0, 8)]
Length    : 8 moves
Cost      : 22
Expanded  : 25
Max frontier: 4
Time      : 0.0337 ms
Map with path:
S*****G
.#####.
.....
.###.###.
.....
```

```
Algorithm : UCS
Path      : [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (1, 8), (0, 8)]
Length    : 12 moves
Cost      : 12
Expanded  : 30
Max frontier: 4
Time      : 0.0503 ms
Map with path:
S~~~~~G
*#####*
*****
.###.###.
.....
```

```
Algorithm : A*
Path      : [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (1, 8), (0, 8)]
Length    : 12 moves
Cost      : 12
Expanded  : 15
Max frontier: 7
Time      : 0.0315 ms
Map with path:
S~~~~~G
*#####*
*****
.###.###.
.....
```

MAP C

The goal is walled off, so no path exists and each algorithm reports "No path found".

```
Algorithm : BFS
No path found
Nodes expanded: 13
```

```
Algorithm : UCS
No path found
Nodes expanded: 13
```

```
Algorithm : A*
No path found
Nodes expanded: 13
```

5. Comparison and Conclusion

Which found the cheaper path?

UCS and A* always return the cheapest path. BFS only minimises the number of moves. On MAP_A all three tie at cost 15. On MAP_B, BFS takes the shortest route (8 moves) but pays 22, because it crosses seven rough cells, while UCS and A* take a longer route (12 moves) that costs only 12. The shortest path is therefore not the cheapest path.

Which expanded fewer nodes?

Among the two cost-optimal algorithms, A* expanded fewer nodes: 21 against 28 on MAP_A and 15 against 30 on MAP_B. The Manhattan heuristic pulls the search toward G, so A* skips cells that UCS must check. The trade-off is a slightly larger peak frontier on MAP_B (7 against 4). On MAP_C all three expand the same 13 cells because there is no goal to be guided toward.

	BFS	UCS	A*
Type	Uninformed	Uninformed	Informed (Manhattan)
Frontier order	Fewest moves (FIFO)	Lowest g	Lowest g + h
Cheapest path?	No (fails on MAP_B)	Yes	Yes
Nodes expanded	A 28 B 25 C 13	A 28 B 30 C 13	A 21 B 15 C 13
Max frontier	A 5 B 4 C 3	A 4 B 4 C 3	A 5 B 7 C 3

Conclusion

For this robot, **A* is the best choice**. The robot pays by terrain cost rather than by number of moves, so BFS is unsuitable: it can return an expensive path, as MAP_B shows. UCS and A* both guarantee the cheapest path, but A* reaches it with less search effort, and Manhattan distance is admissible and consistent here because every move costs at least 1. On these small maps the run times are tiny for all algorithms, but on larger warehouses the smaller number of expanded nodes makes A* the more scalable option. All three algorithms also handle the unreachable-goal case (MAP_C) correctly.

Source Code

The full source code (the notebook with commented code and run instructions) is available on GitHub: <https://github.com/kirogames0/Intelligent-task.git>